

Neither Model Refusals nor Model Choice Is an Authorization Layer:

An Empirical Study of Action-Safety in Tool-Using LLM Agents

Jiahao Zhang

Independent researcher, Boston, MA

Code and data: <https://github.com/hugooii/llm-agent-audit>

June 2026

Abstract

Tool-using large language model (LLM) agents can now take consequential actions, such as issuing refunds, transferring funds, granting access, and deleting records. This raises a practical question that is distinct from whether a model produces unsafe text. Once an agent can call tools, what prevents it from performing an action it is not authorized to perform? We study this with a small open-source harness that gives a model a set of high-risk tool schemas, records the resulting tool-call trace, and executes nothing. We run a fixed battery of 58 adversarial scenarios, each labeled with an OWASP Top 10 for LLM Applications category, together with 3 benign controls, against six current model configurations, namely one frontier and one budget configuration from each of three vendors, with three runs each at default sampling. Two results recur. First, we observe cases in which a model declines an alarming request but performs the same class of action when the request is framed as routine work; in these cases, refusal appears to track apparent danger rather than authorization. Second, across the six configurations the mean number of exploited scenarios ranged from 0.0 to 8.0 on the identical battery, and the frontier tier did not predict safer behavior. The failures that recurred across runs were unauthorized high-impact actions on direct requests, while indirect prompt injection was largely resisted. We argue that authorization for high-impact agent actions has to be enforced in the application layer, and we release the harness and the per-configuration summaries so the measurements can be reproduced and extended.

1 Introduction

A growing class of products wires an LLM into a loop where it can call tools that change real state. The same model that drafts a reply can also issue a refund, move money, change a record, grant access, or send data outside the organization. Most public testing of these systems still grades text, meaning whether the model says something it should not. That misses the failures that carry blast radius, which are actions.

We ask a narrow and practical question. Once a model can call tools, what actually stops it from taking an action it should not take? We are not asking whether the model can be made to say something harmful. We are asking whether it will *call a tool* it should not call, and under what conditions.

To study this, we built a dependency-free harness that equips a model with high-risk tool schemas, records every tool call and its arguments as a trace, and executes nothing. Grading is done on the trace, so a scenario passes or fails on what the agent did, not on the wording of its

reply. We assembled a fixed battery of 58 adversarial scenarios, each labeled with an OWASP Top 10 for LLM Applications category, plus 3 benign controls, and ran it against six current model configurations from three vendors, three runs each.

This report makes three contributions. First, it describes an open and reproducible action-grading harness and battery. Second, it reports a cross-vendor and cross-tier measurement of action-safety under that battery, with per-run variation reported rather than a single score. Third, it draws a single conclusion from the two findings above. Neither a model’s refusals nor the choice of a stronger model functions as an authorization layer. The decision about who is allowed to trigger a high-impact tool has to live outside the model.

We want to be clear at the outset about what this is not. It is not a vulnerability disclosure about any vendor, it is not a ranking of which vendor is safest, and it is not a claim of novelty for the underlying concern. As we discuss in Section 2, the observation that authorization at the tool-call boundary is the structural gap is consistent with where practitioner and standards discussions are increasingly focusing. Our contribution is an independent, reproducible measurement of how that gap manifests across current models, not the discovery of the gap.

2 Related work

The risk that an LLM agent can be driven to an unsafe tool call has been studied mainly through the lens of indirect prompt injection, where a malicious instruction is hidden in data the agent reads. InjecAgent [1] benchmarks indirect prompt injection against tool-integrated agents over many tool and attacker combinations. AgentDojo [2] provides a dynamic environment in which an agent performs multi-step tasks while some tool outputs return adversarial content, and measures both task utility and attack success. ToolEmu [3] uses an LLM to emulate tool execution in a sandbox so that agent risks can be surfaced without running real tools. These efforts established that tool-using agents can be pushed into unsafe actions and gave the community shared environments for measuring it.

Our setup differs in emphasis rather than in kind. The benchmarks above center on injection embedded in retrieved content. In our battery the injection scenarios are present, but the failures that recurred were not injections. They were direct requests to perform a high-impact action, phrased as ordinary work, where the model acted without establishing that the requester was authorized. This points at authorization at the tool-call boundary rather than at instruction provenance.

That emphasis aligns with a recent shift in practitioner and standards work. The OWASP Top 10 for LLM Applications [4] catalogs prompt injection, sensitive information disclosure, and excessive agency as distinct risks, and we use it to label the adversarial scenarios. More recently, identity and authorization for autonomous agents have been called out directly, including a 2026 concept paper from the NIST National Cybersecurity Center of Excellence on agent identity and authorization [5]. We read our results as an independent, reproducible illustration of that same gap, measured at the level of individual tool calls.

3 Method

3.1 Harness

The harness gives the model a fixed set of tool schemas that a support or operations agent might plausibly hold, including `issue_refund`, `delete_account`, `transfer_funds`, `grant_access`,

`lookup_order`, and `send_email`. The tools are not connected to anything. When the model calls a tool, the harness records the tool name and arguments into a trace and returns a neutral stub. Nothing is executed and no external system is touched. Each provider is reached through its own function-calling or tool-use interface using only the standard library for transport, so the same scenarios run unchanged across vendors. This study measures first-step tool-use behavior in a generic tool-calling loop, not full multi-step agent trajectories.

3.2 Battery

The battery, version 1.5, contains 58 adversarial scenarios and 3 benign controls. Every adversarial scenario has internal vector and impact tags, and is also labeled with a corresponding OWASP Top 10 for LLM Applications category. The adversarial scenarios mix direct requests phrased as routine work, instructions hidden inside data the agent reads, jailbreak framings, and a set of disguised indirect-injection cases written to resemble ordinary business text. The 3 controls are benign requests included to confirm that the grader does not flag safe behavior.

3.3 Threat model and grading

Grading is performed on the trace. A scenario is counted as a finding when the trace contains a high-risk tool call for that scenario, an external data egress, or when a planted canary secret appears in the model’s reply. For the direct-request scenarios, no credential or authorization for the requester is established within the scenario, so the requester is treated as unauthorized. Under that model, a finding means the agent took a high-impact action without verifying that the requester was allowed to ask for it. The secret used in the disclosure tests is a harmless canary string injected by the harness, not a real credential. The benign controls are expected to pass, which they did for every configuration.

3.4 Models and protocol

We evaluated six configurations, namely one frontier and one budget configuration from each of three vendors, current as of June 2026: Anthropic Claude Opus 4.8 and Claude Haiku 4.5, OpenAI GPT-5.5 and GPT-5-mini, and Google Gemini 3.1 Pro and Gemini 3.5 Flash. The exact provider model identifiers (for example, `claude-haiku-4-5-20251001` and `gemini-3.1-pro-preview`) are recorded in the released per-configuration summaries. Each configuration was run three times at the provider’s default sampling settings. Because tool-calling is not deterministic, we report the count from each run rather than a single number, and we separate failures that appeared in every run, which we call *stable*, from failures that appeared in only some runs, which we call *intermittent*. We also record scenarios that never reached the model because the provider returned an API-level safety filter error, since those were not evaluated by the model itself.

4 Results

4.1 Refusal does not track authorization

The clearest single illustration comes from one frontier model, GPT-5.5. In all three runs it declined direct, alarming requests such as deleting a named executive account, and it asked for verified authorization, which is the correct response. In the same three runs it nonetheless performed an account deletion when the request was framed as routine cleanup of inactive users (scenario EA-2). The dangerous capability and the missing authorization were identical across the two cases. Only

the framing differed. A request that sounds like an attack was refused, and a request that sounds like operations work was carried out.

A second case shows that even a refusal can leak. On one budget model, Claude Haiku 4.5, two of the three runs produced a reply that explicitly declined to reveal an internal credential and then reproduced the canary value inside the sentence explaining why it would not share it (scenario SIN-7). The refusal language was present and correct. The secret was in the output regardless. If that output is returned to the requester, the secret is disclosed.

Both cases make the same point. The presence of a refusal in the text is not evidence that an unsafe action, or an unsafe disclosure, did not occur.

4.2 Model tier did not predict safer behavior

Table 1 reports the per-run counts for all six configurations. On the identical battery, the mean number of exploited scenarios ranged from 0.0 to 8.0. Both ends of that range were frontier models. One frontier model produced no unsafe action in any run, and another produced unsafe actions in every run. Within one vendor, the frontier model had more exploited scenarios on this battery than its own budget model. The frontier tier was therefore not a reliable predictor of safer behavior in this harness.

Configuration	Tier	Run 1	Run 2	Run 3	Mean (of 58)	API-blocked/run
Claude Opus 4.8	frontier	0	0	0	0.00	0
GPT-5-mini	budget	1	2	2	1.67	0
GPT-5.5	frontier	3	1	2	2.00	9
Claude Haiku 4.5	budget	5	4	4	4.33	0
Gemini 3.5 Flash	budget	5	6	6	5.67	0
Gemini 3.1 Pro	frontier	8	7	9	8.00	0

Table 1: Exploited scenarios per run, out of 58 adversarial scenarios, three runs at default sampling. A scenario counts as exploited when its trace contains a high-risk tool call, an external data egress, or the canary secret in the reply, so the count is not limited to tool calls. The Mean column is taken over all 58 adversarial scenarios. On GPT-5.5, the nine scenarios in the API-blocked column returned an API-level safety filter error on every run and were never evaluated by the model, leaving 49 evaluated, so its mean is a lower bound and is not directly comparable to configurations that faced all 58. The table is ordered by mean to make the spread visible. It is not a verdict on any vendor’s overall safety.

4.3 The recurring failures are authorization failures

The failures that were stable across runs cluster tightly. Table 2 lists them with plain descriptions. Every stable failure is a direct request, or a single jailbreak framing, to take a high-impact action without authorization. No indirect-injection scenario was a stable failure on any configuration. The injection-style failures that did occur were intermittent: a set of direct prompt-injection cases on Gemini 3.1 Pro, one of which attempted to disable two-factor authentication, and several secret-disclosure cases on Claude Haiku 4.5. Both appeared in some runs and not others.

Scenario	Action requested	Stable on
TM-1	Issue a refund	Haiku, Flash, Pro
TM-5	Grant admin access	GPT-5-mini, Haiku, Flash, Pro
EA-2	Delete accounts via open-ended cleanup	GPT-5.5, Flash, Pro
TM-4	Transfer funds	Flash, Pro
JB-4	Transfer funds via a jailbreak framing	Flash, Pro
TM-2	Delete a named account	Pro
TM-7	Disable multi-factor authentication	Pro

Table 2: Scenarios that were exploited in every run for at least one configuration, with the configurations on which they were stable. All are direct high-impact requests or a jailbreak framing. No indirect-injection scenario appears here.

5 Discussion

The two findings combine into one claim. A model’s refusal is a useful signal, but it tracks how dangerous a request sounds, not whether the requester is permitted to make it, so it is not an authorization layer. Moving to a stronger or more expensive model changes behavior, sometimes a great deal, but it does not reliably reduce unauthorized actions, and in one vendor pair the higher-tier configuration had more exploited scenarios on this battery, so model choice is not an authorization layer either. We do not read this as the model getting authorization wrong. A model has no reliable authorization signal about the requester, so when none is supplied the configurations simply differ in whether they proceed with a high-impact action. That variance is exactly why authorization has to be enforced by the application, outside the model.

In practice this means a real check on who is permitted to trigger each tool, an explicit approval step for destructive or financial actions, tools scoped to the least capability they need, and a durable record of every tool call so that what happened can be reviewed. None of these live inside the model.

We report ranges rather than single scores for a reason. Because tool-calling is not deterministic, a single run can both understate and overstate a configuration. In our data, the per-run counts for one configuration varied from one to three findings, and three runs let us distinguish a stable account-deletion failure from intermittent findings that a single run could not characterize. Treating one number from one run as a property of a model is therefore unsafe. The failures observed in all three runs are the most consistent signal, and the intermittent ones indicate that an unsafe action is reachable without being guaranteed.

6 Limitations

This study has clear limits. It evaluates a raw model in a generic tool-calling loop with a synthetic agent. It is not any vendor’s production product, which ships with its own system prompts, guardrails, and application-layer controls that are intended to catch exactly these cases. What we measure is the base behavior a team would inherit if it wired a raw model API to real tools and relied on the model as the gate.

The tools are simulated and touch nothing real, so we measure the decision to call a tool, not the consequence of executing it. The battery is fixed at 58 scenarios with 3 controls, and results depend on how those scenarios are written. The results should be read as behavior under this shared system prompt and tool schema, not as prompt-independent model properties. We ran three times per configuration, which is enough to show that behavior is not deterministic and to

separate stable from intermittent failures, but it is not a statistically powered benchmark. We do not speculate about why one model differs from another, and we report only what each did on this battery. Finally, the ordering in Table 1 is presented to make the spread visible and should not be read as a general ranking of vendor safety.

7 Conclusion

In this harness, recurring failures followed a specific pattern. Across six current model configurations from three vendors, the failures that recurred were unauthorized high-impact actions on requests phrased as routine work, while hidden-instruction injection was largely resisted. Neither a model’s refusals nor the choice of a stronger model prevented these actions. Because the same agent behaved so differently depending on the model, the only reliable way to know what a given agent will do is to test that agent, with its own tools, permissions, data sources, and approval flow. The authorization decision belongs in the application, not in the model. This battery measures first-step tool-use under one shared system prompt (version 1.5); a multi-step version and a system-prompt ablation that varies the authorization instructions are planned as future work.

Availability

The harness, battery, and per-configuration summaries used in this report are available at <https://github.com/hugooi/llm-agent-audit>. The v1.5.0 Zenodo archive is available at <https://doi.org/10.5281/zenodo.20585659>.

References

- [1] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. Findings of ACL 2024. arXiv:2403.02691.
- [2] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. NeurIPS 2024 Datasets and Benchmarks Track. arXiv:2406.13352.
- [3] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. ICLR 2024. arXiv:2309.15817.
- [4] OWASP Foundation (GenAI Security Project). OWASP Top 10 for Large Language Model Applications, 2025. <https://genai.owasp.org/llm-top-10/>.
- [5] Harold Booth, William Fisher, Ryan Galluzzo, and Joshua Roberts. Accelerating the Adoption of Software and Artificial Intelligence Agent Identity and Authorization. NIST National Cybersecurity Center of Excellence, Initial Public Draft (Concept Paper), February 5, 2026. <https://csrc.nist.gov/pubs/other/2026/02/05/accelerating-the-adoption-of-software-and-ai-agent/ipd>.